

RMDB 数据库系统设计与实现说明

得特贝斯队 · 2026 年 8 月

摘要

我们以 RMDB 教学数据库框架为基础，围绕关系型数据库从 SQL 接入到事务持久化、从页面存储到崩溃恢复的完整链路进行了持续实现、补全与重构。系统使用 C++17 编写，对外提供 Wire v3 通信协议，内部形成了由 SQL 前端、查询计划与执行、事务与并发控制、记录与索引、缓冲池与磁盘、预写日志与恢复组成的完整数据库内核。

我们的设计重点并不是针对某几条固定 SQL 建立特殊路径，而是围绕数据库中反复出现的通用成本进行结构性优化：查询阶段尽量减少无效扫描；事务写入阶段先暂存逻辑修改，避免失败事务过早改变共享存储；提交阶段把记录修改按堆表数据页、索引修改按 B+ 树叶页进行批量应用；并发控制将全局共享状态拆分为更细粒度的局部状态；日志提交通过批量持久化摊薄同步开销；崩溃恢复则使用固定缓冲区顺序扫描日志，减少大量零散磁盘读取。

最终系统形成了“计划优化 → 事务私有暂存写 → 冲突校验 → 页级/叶级批提交 → 预写日志持久化 → 提交发布 → 崩溃恢复”的统一执行链。在 32 客户端 TPC-C 兼容负载下，最终正式排名吞吐达到 30,741.2 NewOrder/min，相较早期 32 客户端基线 418 NewOrder/min 提升约 73.5 倍；同时保持功能测试、事务一致性、提交持久化和强制终止后的崩溃恢复正确。

目录

1	设计目标与总体思路	5
1.1	设计目标	5
1.2	设计原则	5
2	系统总体架构	6
2.1	模块划分	6
2.2	一条请求的完整运行链路	6
2.3	运行上下文	7
3	SQL 请求处理与查询执行	7

3.1	SQL 前端	7
3.2	计划到执行器	7
3.3	拉取式执行模型	8
3.4	顺序扫描	8
3.5	索引扫描	9
3.6	连接与聚合	9
4	存储管理	9
4.1	页面式存储	9
4.2	堆表记录组织	10
4.3	记录层职责划分	10
4.4	缓冲池	10
4.5	64 分片页表与帧级保护	11
4.6	页面守卫	11
4.7	页面替换	11
5	B+ 树索引设计	11
5.1	索引基本结构	11
5.2	复合索引	11
5.3	唯一性维护	12
5.4	局部门锁与逐层保护	12
5.5	同叶批写	12
6	查询优化设计	13
6.1	优化目标	13
6.2	谓词规范化与去重	13
6.3	单表谓词下推	13
6.4	等值关系传播	13
6.5	最长连续索引前缀	13
6.6	连接顺序	13
6.7	语义回查	14
7	事务与并发控制	14
7.1	事务生命周期	14
7.2	三种隔离级别	14
7.3	多版本状态	15
7.4	256 分片并发状态	15
7.5	冲突尽早发现	15
7.6	连续可见提交前缀	15
8	数据修改与提交机制	16
8.1	为什么不在语句阶段立即物理写	16

8.2	事务私有暂存写	16
8.3	读己之写	16
8.4	提交准备	17
8.5	堆表按页批量应用	17
8.6	索引按页批量应用	17
8.7	提交阶段并发上限	18
8.8	轻量中止	18
9	预写日志与崩溃恢复	18
9.1	预写日志的作用	18
9.2	日志序列号	18
9.3	提交日志与组提交	18
9.4	检查点	19
9.5	启动恢复流程	19
9.6	分析阶段	19
9.7	重做阶段	19
9.8	未提交事务不做完整物理重做	20
9.9	撤销阶段	20
9.10	索引恢复	20
9.11	日志顺序扫描器	20
9.12	定向恢复刷盘	21
10	网络协议、预处理语句与批执行	21
10.1	Wire v3 请求模型	21
10.2	普通流式执行	22
10.3	预处理语句	22
10.4	参数绑定	22
10.5	批执行	22
11	性能重构与关键优化	22
11.1	早期并发退化	22
11.2	重构总原则	23
11.3	事务级合并优于单次操作微优化	23
11.4	局部快速路径必须带安全回退	24
11.5	性能和恢复共同演化	24
12	正确性设计	24
12.1	事务原子性	24
12.2	表与索引一致性	24
12.3	未提交数据不可见	24
12.4	无缺口快照	25
12.5	预写日志先行	25

12.6	提交成功之前必须持久化	25
12.7	恢复必须可重复	25
12.8	浮点语义	25
12.9	装载与批处理仍属于事务语义	25
13	性能结果与资源权衡	25
13.1	吞吐演进	25
13.2	性能提升的来源	26
13.3	内存代价	26
13.4	剩余性能瓶颈	26
14	当前局限与后续方向	26
14.1	基于统计信息的成本优化器	26
14.2	增量或模糊检查点	27
14.3	多版本垃圾回收	27
14.4	更完善的缓冲替换策略	27
14.5	更细粒度的后台持久化	27
14.6	索引结构维护	27
15	总结	27

1 设计目标与总体思路

1.1 设计目标

我们将系统目标概括为四点：**正确、完整、高并发、可恢复**。

第一，数据库必须保持完整的事务语义。任何一次插入、更新或删除都不能只修改表而漏改索引，也不能在事务失败后留下部分结果。对于已经向客户端确认提交成功的事务，即使数据库进程随后被强制终止，其结果仍必须在重启后存在。

第二，系统需要具备完整的关系型数据库执行链。SQL 不能直接映射到某个手写函数，而是必须经过词法与语法分析、语义分析、执行计划生成、执行器树构造，再进入记录、索引和事务层。这种分层使 SQL 语义、物理访问路径和事务控制能够相互独立演进。

第三，我们希望系统在高并发下仍能扩展。早期版本的主要问题并不是单条 SQL 太慢，而是大量无关事务争用全局共享状态，失败事务又会先完成大量物理操作，导致客户端数增加后吞吐反而下降。因此后续设计的核心是**减少不必要的共享、把冲突尽量提前、把真正昂贵的物理操作尽量批量执行**。

第四，系统必须把崩溃恢复作为正常设计的一部分，而不是运行结束后的补丁。事务提交、日志持久化、数据页刷盘、版本发布和恢复行为必须共同构成一个闭环。

1.2 设计原则

1.2.1 分层而不割裂

我们将数据库拆分为协议接入、SQL 前端、计划与执行、事务与并发、记录与索引、缓冲与磁盘、日志与恢复等模块。各模块职责独立，但通过明确的数据结构衔接。例如执行器不直接操作磁盘文件，而是通过记录管理层获取记录；记录管理层不自行完成事务可见性判断，而是通过运行上下文与事务层协作。

1.2.2 优化通用机制，而不是识别固定负载

我们允许计划缓存、复合索引、批执行、页批写和分片并发等优化，但这些机制的触发依据必须来自 SQL 结构、索引元数据、页面归属和运行时状态，而不能依赖固定表名、固定语句编号或固定客户端数量。

1.2.3 正确性先于性能

我们把事务原子性、隔离性、持久性和恢复一致性视为所有优化的边界。任何局部优化如果造成部分提交可见、索引和表不一致、提前返回提交成功或恢复后状态错误，即使吞吐更高也不保留。

1.2.4 延迟共享，集中物理修改

写事务在语句执行阶段尽可能停留在事务私有状态中，只有通过必要校验并进入提交阶段后，才将修改集中应用到堆表和索引。这样可以同时降低失败事务的回滚成本和共享数据结构的竞争时间。

1.2.5 持久化先于对外可见

系统遵循“**先完成物理应用和日志持久化，再发布提交状态**”的原则。提交时间戳的分配与真正对新快照可见的发布过程分离，避免新事务观察到只完成一半的提交。

2 系统总体架构

2.1 模块划分

我们将当前 RMDB 内核划分为以下七个主要部分：

模块	主要职责	主要目录/组件
协议与连接	接收请求、解析 Wire v3 报文、维护连接级预处理语句字典、组织返回结果	<code>network/</code> 、连接会话与请求分发组件
SQL 前端	词法分析、语法分析、语义检查、列和表绑定	<code>parser/</code> 、 <code>analyze/</code>
查询计划	生成逻辑/物理计划，进行谓词处理、索引选择和连接顺序优化	<code>optimizer/</code> 、 <code>planner.cpp</code>
执行引擎	将计划转换为执行器树，驱动扫描、连接、聚合、投影及数据修改	<code>portal.h</code> 、 <code>execution/</code>
事务与并发	管理事务生命周期、快照、多版本状态、冲突检测、暂存写和提交发布	<code>transaction/</code>
存储与索引	管理堆表记录、记录标识、B+ 树索引、缓冲页面和磁盘文件	<code>record/</code> 、 <code>index/</code> 、 <code>storage/</code> 、 <code>replacer/</code>
日志与恢复	维护预写日志、提交持久化、检查点以及启动时的分析/重做/撤销	<code>recovery/</code>

2.2 一条请求的完整运行链路

一条普通查询从客户端进入系统后，大致经历以下过程：

```
客户端
↓
Wire v3 协议层
↓
语法分析 Parser
↓
语义分析 Analyze
↓
Query: 已经完成表、列、类型和条件绑定的查询描述
↓
Planner / Optimizer: 选择访问路径并生成 Plan
↓
Portal: 把 Plan 递归转换为 Executor 执行器树
↓
Executor: 扫描、过滤、连接、聚合、投影或数据修改
↓
事务可见性与冲突控制
```

```
↓
Record / B+Tree
↓
Buffer Pool
↓
Disk
```

对于写事务，执行器得到目标记录后不会在所有隔离级别下都立即进行物理写入。在快照隔离和可串行化路径中，修改首先进入事务私有暂存写集合，随后在提交阶段完成冲突校验、堆表与索引物理应用以及日志持久化。

2.3 运行上下文

为了避免在执行器、记录层和事务层之间不断传递大量独立参数，我们使用运行上下文 `Context` 保存一次 SQL/事务执行所需的共享环境，例如当前事务、事务管理器、锁管理器、日志管理器、隔离级别和结果输出状态。

我们把它理解为执行链中的“运行时环境”。记录标识 `Rid` 说明“数据在哪里”，而运行上下文说明“当前事务是谁、它可以看到什么、是否需要加锁或写日志”。这种设计使底层记录接口既可以服务普通读取，也可以根据当前事务语义完成不同的可见性和并发行为。

3 SQL 请求处理与查询执行

3.1 SQL 前端

SQL 前端分为语法分析和语义分析两个阶段。

语法分析负责把 SQL 文本转换为抽象语法树。此时系统知道语句的结构，例如它是 `SELECT`、`UPDATE` 还是 `INSERT`，包含哪些表、列、表达式和条件，但还没有确认这些名称在当前数据库中是否真实存在。

语义分析进一步结合系统目录完成表名、列名和类型检查，将语法结构转换为可以被优化器直接使用的 `Query`。我们在这一阶段消除诸如“列不存在”“列名歧义”“类型不兼容”等问题，使后续计划生成只处理已经合法的查询。

因此三者的职责可以概括为：

```
AST: SQL 的语法结构
Query: 完成语义绑定后的查询意图
Plan: 数据库准备如何执行这个查询
```

3.2 计划到执行器

计划生成阶段根据查询结构选择顺序扫描、索引扫描、连接方式、过滤条件和投影列等物理执行方式。生成的计划是一棵树，而不是直接执行数据访问。

`Portal` 负责把计划树递归转换为执行器树。例如一个查询：

```
SELECT name
FROM student
WHERE id = 10;
```

可能形成：

```
ProjectionExecutor
  ↓
FilterExecutor
  ↓
IndexScanExecutor / SeqScanExecutor
```

这种结构使每个执行器只关心自己的职责，扫描执行器负责产生记录，过滤执行器负责判断条件，投影执行器只保留需要输出的列。

3.3 拉取式执行模型

执行引擎采用典型的拉取式迭代执行模型。上层执行器不断向下层“要一条记录”，下层再继续向更底层取数据。

核心接口可以理解为：

- `beginTuple()`：把执行器定位到第一条可用记录；
- `Next()`：返回当前记录；
- `nextTuple()`：向下一条记录推进；
- `is_end()`：判断扫描是否结束。

这种模型结构简单，执行器组合灵活。我们在此基础上又增加页级批扫描能力，使存储层可以一次返回一批候选记录，减少逐记录函数调用和重复页面固定/释放的成本。

3.4 顺序扫描

顺序扫描执行器并不自己遍历磁盘字节。实际路径是：

```
SeqScanExecutor
  ↓
RmScan
  ↓
找到下一个有效 Rid
  ↓
RmFileHandle 根据 Rid 读取记录
  ↓
BufferPoolManager 获取页面
  ↓
从 Page 的目标 Slot 复制出 RmRecord
```


↓
事务层进行可见性判断
↓
执行 WHERE 条件

RmScan 的职责很单纯：遍历记录页的位图，找到下一个有效槽位并产生 Rid。它并不知道 SQL 中是否存在 `age > 20` 这样的条件。SQL 条件属于执行层，而页面和槽位遍历属于记录层，这种边界能够避免存储层与 SQL 语义耦合。

3.5 索引扫描

索引扫描首先在 B+ 树中根据键或键范围得到候选 Rid，随后仍然回到堆表读取真实记录，并进行事务可见性与完整谓词检查。

我们坚持“索引只负责缩小候选集合，不替代最终语义判断”。这样即使索引中存在历史项、候选范围较宽，或者复合索引只使用了部分前缀，最终结果仍由真实记录和完整条件决定。

3.6 连接与聚合

执行引擎实现普通嵌套循环连接以及可利用内表索引的索引嵌套循环连接。对于外表的每条记录，如果连接键可以匹配内表索引，我们直接通过 B+ 树定位内表候选记录，而不是反复完整扫描内表。

聚合执行支持 COUNT、SUM、AVG、MIN、MAX 以及分组。聚合阶段先根据 WHERE 得到输入集合，再按照 GROUP BY 形成分组并计算聚合结果，HAVING 则作用于聚合后的分组结果。

4 存储管理

4.1 页面式存储

我们以 4KB 页面作为磁盘和缓冲池之间的基本 I/O 单位。数据库不会为了读取一条几十字节的记录直接对磁盘发起几十字节随机 I/O，而是把包含该记录的整个页面调入缓冲池。

Page 同时表示一个逻辑数据库页在内存中的缓存对象，内部保存：

- 页面标识 PageId；
- 4KB 页面数据；
- 脏页状态；
- 固定计数 pin_count；
- 页面日志序列号等持久化相关信息。

其中 PageId 由文件描述符和页号共同标识，因此不同表文件中的相同页号不会混淆。

4.2 堆表记录组织

数据库中的 Heap 在这里指**堆表物理数据**，不是 C++ 的堆内存。表记录按照记录页组织，每个页面包含页面头、槽位位图和若干定长记录槽。

一条记录的位置由 Rid 表示：

```
Rid = (page_no, slot_no)
```

因此记录访问被拆成两个问题：先找到记录位于哪个页面，再找到页面中的哪个槽位。

位图负责说明槽位是否被占用。顺序扫描只需检查位图中的有效位，而不需要逐字节判断记录是否存在。

4.3 记录层职责划分

记录管理主要由三个角色完成：

4.3.1 RmManager

RmManager 负责记录文件的生命周期，包括创建、删除、打开和关闭表数据文件。它管理的是“文件”。

4.3.2 RmFileHandle

RmFileHandle 表示一张已经打开的记录文件，负责真正的记录读取、插入、更新和删除，也负责根据 Rid 获取相应页面和槽位。执行器对某张表的物理访问最终会落到对应的 RmFileHandle。

4.3.3 RmScan

RmScan 负责按照页面和位图遍历有效记录位置，主要输出 Rid。因此可以概括为：

```
RmManager：管理记录文件  
RmFileHandle：管理一张已经打开的表  
RmScan：遍历记录位置
```

4.4 缓冲池

缓冲池的作用是把有限内存作为磁盘页面缓存。读取页面时，系统首先查询页面是否已经在缓冲池中：

```
页面已缓存  
→ 增加固定计数  
→ 直接返回  
  
页面未缓存  
→ 选择空闲帧或可淘汰帧  
→ 必要时写回旧脏页  
→ 从磁盘读取新页面  
→ 建立 PageId 到 Frame 的映射
```

一个页面的固定计数大于 0，表示仍有执行路径正在使用它，此时不能被替换器淘汰。

4.5 64 分片页表与帧级保护

早期缓冲池使用较粗粒度的共享保护，多个线程即使访问不同页面也可能竞争同一个临界区。我们将页面映射表拆成 **64 个分片**，按 PageId 哈希到不同分片，每个分片独立保护自己的页面映射。

同时每个缓冲帧维护独立控制状态，包括：

- 元数据互斥保护；
- 页面内容读写保护；
- I/O 状态；
- 代际编号；
- 脏页状态和相关日志序列号。

因此不同页面能够在不同分片和不同帧上并行访问，而只有页面淘汰、空闲帧选择等真正涉及全局资源的步骤仍需要更强的协调。

4.6 页面守卫

为了避免复杂执行路径中忘记释放页面，我们使用页面守卫管理页面生命周期。基本、只读和写入守卫分别封装页面固定、读写保护和释放过程，离开作用域时自动解除保护并执行相应的 `unpin`。

这一机制减少了异常路径和提前返回导致的页面永久固定问题，也让存储代码更容易明确“这一段代码正在读页面还是写页面”。

4.7 页面替换

当前实现采用 LRU 类替换策略维护可淘汰帧。只有固定计数归零的页面才会进入可替换集合。页面再次被访问时从可替换集合移出，页面释放后重新进入候选集合。

我们没有把替换策略与记录层绑定，因此未来可以在不改变上层记录和查询接口的情况下替换为更复杂的缓存策略。

5 B+ 树索引设计

5.1 索引基本结构

B+ 树负责维护“索引键 → 记录位置”的映射。叶子节点保存键和对应 Rid，内部节点保存分隔键和子节点指针。所有真实数据记录仍保存在堆表中，因此索引扫描得到 Rid 后还需要回表读取记录。

B+ 树适合数据库索引的原因在于其扇出大、树高低，并且叶节点有序连接，既能支持等值查找，也适合范围查找和有序遍历。

5.2 复合索引

对于复合索引 (a, b, c)，我们把多个列按索引定义顺序编码成一个连续键。查询能有效使用多长的索引，取决于条件是否覆盖从第一列开始的连续前缀。

例如：

索引: (a, b, c, d)

条件: a=1, b=2, c=3

→ 连续前缀长度为 3

条件: a=1, c=3

→ 因为中间缺少 b, 连续前缀只能使用 a

这也是查询优化器选择索引的重要依据。

5.3 唯一性维护

唯一索引必须在并发环境下保证同一个逻辑键不能被两个事务同时提交。我们不仅检查当前物理 B+ 树中的键, 还结合事务层的唯一键写意向处理尚未提交的并发写。

真正插入索引项之前仍会执行最终唯一性检查。如果快速路径发现叶页状态变化或键归属不再可靠, 则放弃快速路径并回到完整 B+ 树操作。

5.4 局部锁与逐层保护

锁用于保护数据库内部共享数据结构的物理一致性。它和事务锁不同: 事务锁解决“两个事务在逻辑上能否同时修改某对象”, 锁解决“两个线程不要同时把同一个 B+ 树页改坏”。

早期如果使用整棵索引级的大锁, 即使两个线程修改完全不同的叶页也会互相等待。我们改为局部逐层保护: 向下访问子节点时获取必要的子页保护, 当确认祖先不再需要参与结构修改时尽早释放祖先。

发生分裂并同时涉及多个页面时, 我们按照物理页号确定一致的加锁顺序, 减少相反顺序获取页面导致的死锁风险。

5.5 同叶批写

单条索引修改的固定成本包含从根下降到叶页、获取页面、键比较和锁切换。当一个事务需要连续插入多个相近索引键时, 这些键往往落在相同或相邻叶页。

因此我们将待提交的索引修改按键排序并按目标叶页分组:

原方式:

key1 → root → leaf

key2 → root → leaf

key3 → root → leaf

批量方式:

key1、key2、key3 排序

→ 定位 leaf

→ 一次写保护下连续应用多个修改

复用叶页的前提是叶页代际、键范围、容量和结构关系仍然有效。一旦发生叶分裂、键越界或结构版本变化，我们立即结束局部快速路径并重新从安全位置下降，以性能换取之前先验证结构正确性。

6 查询优化设计

6.1 优化目标

我们的优化器首先解决的不是复杂的成本估算，而是一些确定性的高收益问题：**少产生中间记录、尽早利用选择性条件、尽可能使用连续复合索引前缀、避免明显不合理的连接顺序。**

6.2 谓词规范化与去重

同一个逻辑条件可能通过不同路径进入计划。如果重复保留，会在扫描或连接过程中被重复计算。我们首先规范化并去重谓词，为后续下推和索引分析提供统一表示。

6.3 单表谓词下推

只依赖一张表的条件尽可能靠近该表的扫描节点执行。

例如：

```
SELECT ...  
FROM A JOIN B ON A.id = B.a_id  
WHERE B.status = 1;
```

如果先把 A 和完整 B 连接再过滤，会产生大量无意义中间记录。我们会先把 `B.status = 1` 下推到 B 的扫描阶段，再参与连接。

6.4 等值关系传播

如果查询中存在：

```
A.x = B.x  
A.x = 10
```

则可以推导出 `B.x = 10`，从而使 B 也可能使用对应索引。推导条件用于帮助选择访问路径，但不会删除原始连接条件，最终仍重新检查完整 SQL 语义。

6.5 最长连续索引前缀

我们对候选索引计算当前查询能够覆盖的最长连续前缀，并结合过滤条件和已经连接的列为候选访问路径评分。完整键可以进行精确等值查找，真实前缀可以形成范围扫描，无法构成连续前缀时则回到更一般的扫描方式。

6.6 连接顺序

多表连接的中间结果大小高度依赖连接顺序。我们综合单表过滤条件、可利用索引前缀和表之间的连接关系，优先选择能尽早缩小候选集合的表进入连接树。

当前方法仍属于启发式规划，而不是完整的统计信息驱动成本优化器，但相比固定连接顺序，它能够显著降低 TPC-C 多表查询和一致性检查中的中间数据量。

6.7 语义回查

所有优化都遵循一个原则：**索引、传播条件和计划重排只改变候选获取方法，不改变最终判定规则**。回表后系统仍会执行事务可见性判断和全部原始过滤/连接条件，从而保证不同合法访问路径得到一致的外部结果。

7 事务与并发控制

7.1 事务生命周期

我们把一个写事务概括为以下阶段：

```
BEGIN
↓
建立事务状态 / 快照
↓
执行 SQL
↓
读取已提交版本 + 合并本事务暂存写
↓
登记 INSERT / UPDATE / DELETE 暂存修改
↓
冲突与可串行化校验
↓
提交阶段物理应用
↓
预写日志持久化
↓
发布提交版本
↓
释放事务资源
↓
COMMIT ACK
```

如果过程中发现冲突或执行错误，则进入中止流程，撤销已经产生的物理影响，并丢弃未应用的事务私有状态。

7.2 三种隔离级别

系统支持读已提交、快照隔离和可串行化三种隔离语义。

7.2.1 读已提交

每条语句读取执行时已经提交的数据，主要采用传统锁式和记录写集机制完成并发控制。

7.2.2 快照隔离

事务开始时获得一个合法已发布提交前缀作为读时间点。在事务生命周期内，读取同一快照中的数据。本事务产生的新修改通过私有暂存写覆盖到读取结果中，从而实现“读己之写”。

对同一记录发生并发写时执行写写冲突检测，陈旧快照上的写事务不能覆盖更新后的提交结果。

7.2.3 可串行化

在快照隔离基础上继续记录读写依赖和谓词范围依赖，检测可能形成不可串行化结果的危险结构。当系统无法安全提交某个事务时，将其中止而不是暴露写偏斜等异常。

7.3 多版本状态

多版本并发控制保存的是记录在不同事务时间点上的逻辑状态。当前物理堆表记录并不能独自回答“某个旧快照应该看到什么”，因此事务读取过程需要结合版本链和事务快照进行判断。

一个尚未提交的新版本处于待提交状态。创建它的事务可以通过自己的暂存状态看到它，其他事务则继续看到合法的已提交旧版本。事务提交后该版本才进入公共可见范围；事务中止则直接丢弃。

7.4 256 分片并发状态

如果所有记录版本、唯一键意图和冲突状态都由一把全局互斥锁保护，那么访问完全不同记录的事务也会互相等待。我们将多版本核心状态按资源键哈希为 **256 个分片**，不同记录和不同唯一键意图尽可能落到不同的局部临界区。

这样做并不会消除真正的热点冲突。同一个库存记录仍必须正确竞争，但不同仓库、不同商品或不同记录之间不再因为一个全局状态锁被迫串行。

7.5 冲突尽早发现

高并发系统中一个重要成本来自“注定失败的事务还在继续执行”。如果事务已经基于陈旧快照，并且所写记录已经被其他事务成功提交，那么继续完成后续查询、页面访问和索引维护只会浪费资源。

因此我们尽可能在语句执行和提交准备阶段发现写冲突、唯一性冲突以及可串行化冲突，并尽早中止事务。失败事务越早结束，对缓冲池、索引和日志的压力越小。

7.6 连续可见提交前缀

并发事务的提交时间戳分配顺序与物理完成顺序可能不同。例如事务 A 获得时间戳 100，事务 B 获得 101，但 B 先完成物理提交。

新快照不能因为 101 已经完成就直接读到 101，否则可能跨过尚未完成的 100，形成有缺口的提交历史。我们因此区分：

- **提交时间戳分配**：给事务一个逻辑顺序号；
- **连续可见前缀**：只有从之前可见位置开始连续完成的提交才能推进。

新事务捕获的是连续已发布前缀，而不是简单的“当前最大时间戳”。

8 数据修改与提交机制

8.1 为什么不在语句阶段立即物理写

早期写路径中，INSERT、UPDATE、DELETE 会较早修改堆表和索引。如果事务执行到后面才发现冲突，则需要把之前的堆表和索引修改全部反向恢复。

TPC-C 中写事务往往包含多张表和多个索引，失败成本非常高。因此我们在快照隔离和可串行化路径中引入事务私有写批 TransactionWriteBatch，把语句执行和最终物理提交分离。

8.2 事务私有暂存写

执行器首先记录逻辑变化：

```
stage_insert
stage_update
stage_delete
```

这些操作说明“事务最终想把数据变成什么样”，而不立即把变化暴露给共享堆表和索引。

例如物理记录当前为：

```
quantity = 60
```

同一事务连续执行：

```
60 → 55
55 → 48
```

写批可以合并为最终净变化：

```
60 → 48
```

如果一条记录先在本事务中插入，随后又被本事务删除，则这两次逻辑操作可以直接相互抵消，不需要产生真实堆表和索引写入。

8.3 读己之写

暂存并不意味着事务自己看不到修改。读取时我们先获取快照中合法的基础记录，再把当前事务自己的暂存写覆盖到结果上。

因此：

物理堆表: money = 100
当前事务暂存: money = 200

当前事务再次读取 → 200
其他事务读取 → 合法已提交版本 100

这使事务内部语义保持正常，同时未提交数据不会污染其他事务的物理视图。

8.4 提交准备

提交前，我们冻结当前事务的写集合，使后续提交过程中写集合不再变化，并完成必要的冲突校验、唯一键意图获取和版本合法性检查。

如果这些检查失败，且事务尚未产生物理副作用，则可以直接丢弃事务私有写批和待提交版本，不需要对整个数据库进行重型回滚。

8.5 堆表按页批量应用

真正物理提交时，我们将写集合按照 PageId 分组。

例如一个事务要修改：

Page 10 / Slot 1
Page 10 / Slot 3
Page 10 / Slot 7
Page 11 / Slot 2

不再对 Page 10 连续执行三次独立的获取、写保护、脏页标记和释放，而是：

获取 Page 10 的写守卫
→ 验证目标 Rid 与旧值
→ 连续应用 Slot 1、3、7
→ 统一更新页面状态
→ 释放

再处理 Page 11

这样把逐记录固定成本摊到一组同页修改上。

8.6 索引按叶页批量应用

索引修改则按键排序并按叶页聚合。一次根到叶的定位和一次叶页写保护可以服务多条插入/删除操作。结构发生变化时，批量快速路径会回退到完整 B+ 树路径。

堆表页批写和索引叶批写共同把事务提交从“很多独立小物理操作”转变为“少量局部批操作”。

8.7 提交阶段并发上限

提交阶段会集中产生堆表写、B+ 树修改和日志写入。如果无限数量事务同时进入这一阶段，可能形成缓冲池争用和日志同步风暴。

因此我们对物理应用阶段设置并发上限，使前台仍具有足够并行度，同时防止大量提交线程同时占用关键存储资源。该限制是资源调度机制，不改变事务冲突和隔离语义。

8.8 轻量中止

事务中止分两种情况：

第一种是修改仍然只存在于私有写批和待提交版本中，没有改变堆表和索引。这种事务只需要清理内存状态、唯一键意图和事务注册信息。

第二种是已经产生物理修改，则仍必须根据写记录进行完整反向恢复，保证堆表、索引和事务元数据都回到合法旧状态。

因此我们的“轻量中止”并不是跳过回滚，而是识别“根本没有物理副作用”的事务，避免为它执行没有意义的全库检查点或单独磁盘同步。

9 预写日志与崩溃恢复

9.1 预写日志的作用

数据页在缓冲池中修改后并不一定立即写回磁盘，因此“事务已经提交”和“数据页已经落盘”是两个不同事件。

我们使用预写日志（WAL）记录事务的插入、删除、更新和提交信息。核心规则是：

对应日志必须先于数据页持久化。

这样即使事务已经成功提交，但对应数据页仍停留在内存中，数据库崩溃后也可以根据日志重新得到正确结果。

9.2 日志序列号

每条日志拥有递增的日志序列号 LSN，用于描述日志顺序和持久化进度。事务日志还保存必要的前驱关系，使系统可以定位一个事务的历史操作。

系统同时维护持久化水位。只有覆盖某个提交记录的日志已经完成稳定化，该事务才满足对外返回提交成功的持久化条件。

9.3 提交日志与组提交

如果每个事务都单独执行一次磁盘同步，存储设备的同步延迟会直接串行进入每次提交。高并发下，线程可能大量排队等待 `fsync`。

我们采用组提交思想：在很短的聚合窗口内收集多个事务需要持久化的日志，由一次日志刷新和稳定化覆盖多个事务。每个等待事务只在持久化水位已经覆盖自己的提交日志后才允许继续返回成功。

因此组提交优化的是同步次数，而不是放宽提交持久化要求。

9.4 检查点

检查点用于建立一个已知一致的持久化位置，减少数据库重启时需要从日志最开始分析的范围。当前检查点仍偏静态：在建立检查点时协调正在执行的事务并刷新必要状态，同时记录可用于下一次启动的重启位置和索引快照信息。

这种实现结构清晰、恢复边界容易验证，但长事务和大规模脏页情况下会增加检查点窗口，因此后续仍有增量/模糊检查点的优化空间。

9.5 启动恢复流程

数据库启动时，恢复流程在监听客户端连接之前完成：

```
打开数据库
↓
分析日志 analyze
↓
根据分析结果初始化日志管理器
↓
重做 redo
↓
撤销 undo
↓
刷新恢复真正影响的数据和元数据
↓
恢复完成
↓
建立监听端口
```

因此恢复阶段任何全表扫描、全索引重建或无意义全量刷盘都会直接影响数据库重启后的就绪时间。

9.6 分析阶段

分析阶段从合法重启位置向后顺序扫描日志，识别：

- 哪些事务已经提交；
- 哪些事务已经中止；
- 哪些事务在崩溃时仍然活动；
- 日志中最后一个完整有效记录的位置；
- 最大日志序列号；
- 是否存在可用检查点及索引快照。

分析结果同时用于初始化新的事务编号和日志管理状态，避免启动后重新从头进行一遍相同的全量日志扫描。

9.7 重做阶段

重做用于确保应该存在的物理状态真正存在。对于已经完成并应当保留的事务，我们根据日志执行幂等安装、删除或更新操作。

当前恢复实现不是简单依赖页面日志序列号比较来跳过所有已经执行过的日志，而是通过恢复专用的幂等记录接口维护物理状态。例如：

- `upsert_record_for_recovery`: 目标记录应该存在时，保证它最终存在且内容正确；
- `delete_record_for_recovery`: 目标记录应该不存在时，保证它最终被删除。

这样即使恢复过程中再次崩溃，下一次启动重复执行同一恢复动作，也不会因为“记录已经存在”或“记录已经删除”而破坏结果。

9.8 未提交事务不做完整物理重做

崩溃时仍活动的事务属于未提交事务。我们在重做阶段记录这些事务的日志位置，但不把它们当成已提交事务完整物理重做。

原因是未提交事务可能和已经提交事务曾经竞争同一个唯一键。如果先把未提交事务的所有动作重新写入 B+ 树，再在撤销阶段反向删除，可能在重做阶段就与已经提交事务产生唯一索引冲突。

因此当前设计是：

已提交事务 → 幂等重做
已经明确中止的事务 → 不复活原修改
崩溃时仍活动的事务 → 登记动作位置，随后进入撤销

9.9 撤销阶段

对崩溃时仍活动的事务，我们按照日志动作的逆序执行反操作：

INSERT → 删除对应记录
DELETE → 恢复删除前记录
UPDATE → 恢复旧值

撤销完成并且物理恢复结果已经持久化后，再为这些事务追加中止日志。这样如果数据库在恢复撤销过程中再次崩溃，下次仍可以安全重复撤销，直到中止状态真正持久化。

9.10 索引恢复

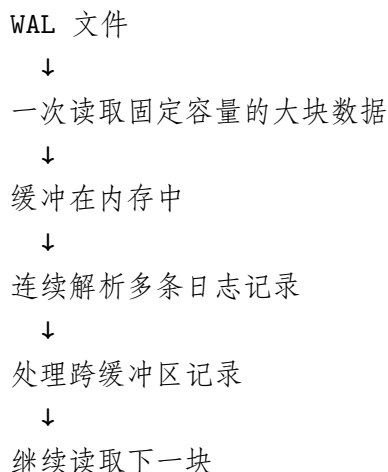
如果检查点提供了可信索引快照，我们在恢复过程中按记录变化增量维护索引。插入索引项时会先检查当前键是否已经正确指向目标 `Rid`；删除时也只有当前索引项确实指向该记录才执行删除。

如果没有可用索引快照，则在恢复后对需要的索引进行重建。我们避免无条件重建所有索引，因为这会直接拖慢启动就绪路径。

9.11 日志顺序扫描器

前台吞吐提高后，单位时间产生的日志量显著增大。早期恢复读取每条日志时分别读取头部和正文，并频繁获取文件状态，大量小 `pread` 系统调用成为恢复瓶颈。

为此我们实现 `LogRecordScanner`：



扫描器同时识别日志末尾不完整记录并停止在最后一个完整位置。分析和重做共享统一扫描方式，分析阶段得到的有效日志末尾和最大日志序列号直接用于日志管理器初始化。

在同一份约 35MiB 日志的本地采样中，恢复相关系统调用由约 **63.5 万次**降低到约 **3,312 次**，减少约 **99.5%**。这项优化本身不提高正常事务吞吐，但保证前台吞吐提升、日志量增大以后数据库仍可以在合理时间内完成恢复。

9.12 定向恢复刷盘

恢复结束后，我们不再无条件遍历和刷新整个数据库。

如果存在可信检查点，则避免对所有表进行完整空闲页链重建；只有确实被恢复过程修改过的表进入定向刷新。索引快照可用时也只维护实际变化的索引。

这样恢复的最后阶段从“全库整理”转变为“只持久化本次恢复触碰过的状态”，缩短了数据库重新接受连接之前的临界路径。

10 网络协议、预处理语句与批执行

10.1 Wire v3 请求模型

数据库对外通过 Wire v3 协议接受请求。协议层负责：

- 报文边界解析；
- 参数类型解码；
- 流式 SQL 执行；
- 预处理语句安装；
- 批量操作执行；
- 查询结果和事务状态编码。

网络层不直接实现 SQL 语义，而是把已经解码的请求交给 SQL 执行服务。

10.2 普通流式执行

普通 SQL 使用流式执行路径。查询结果按照元数据、结果行和结束标记顺序返回；非查询命令返回单一命令完成状态。

功能测试、大结果查询、装载和恢复后检查等路径都可以使用这一通用执行方式。

10.3 预处理语句

TPC-C 等负载会高频重复相同 SQL 模板，只是仓库号、客户号、订单号、商品号等参数不断变化。如果每次都重新进行 SQL 文本解析、语义绑定和计划生成，会浪费大量固定 CPU 成本。

因此我们支持预处理语句集合。服务器在准备阶段保存：

```
statement_id
→ 解析后的语句/执行计划
→ 参数类型
→ 结果结构
```

正式执行时客户端只需发送语句编号和本次真实参数，服务器找到已准备的执行对象后完成参数绑定并访问真实数据库状态。

我们缓存的是“如何执行”，不是“查询结果是什么”。数据发生变化后，同一个准备计划仍然必须读取当前事务真正可见的数据。

10.4 参数绑定

参数不会简单拼回 SQL 字符串再重新解析，而是绑定到已经准备好的语法结构或计划参数槽位。这样既减少重复解析，也避免字符参数再次经过文本拼接路径。

10.5 批执行

EXEC_BATCH 可以在一次请求中携带多个已经准备好的操作及其参数，并严格按照操作顺序执行。

批执行带来的收益主要有两类：

第一，减少网络往返。一个事务中的多个操作可以在更少的请求/响应周期内完成。

第二，给数据库内部提供更清晰的事务级工作集合，使后续暂存写、同页批写、同叶批写和统一结果构造更容易发挥作用。

如果批中某个操作失败并要求自动中止，我们会停止后续操作，先完整回滚当前事务，再返回事务中止结果；不会把失败前收集到的部分查询结果作为成功结果发送出去。

11 性能重构与关键优化

11.1 早期并发退化

早期系统在单客户端下已经可以达到较高执行速度，但客户端数增加后吞吐持续下降：

客户端数	早期吞吐
1	4,497
4	2,427
8	1,095
16	891
32	418

32 客户端吞吐只剩单客户端的大约 9.3%。这说明主要问题不是某条 SQL 的局部计算复杂度，而是并发竞争、失败事务浪费和同步成本随客户端数量快速放大。

我们将问题归纳为四类：

1. **冲突发现太晚**：事务已经完成大量查询和部分物理工作后才发现无法提交；
2. **共享范围太大**：不同事务访问不同数据仍争用相同全局状态；
3. **物理操作太碎**：同页多条更新仍逐记录获取页面，多个相近索引键仍反复从根下降；
4. **持久化太频繁**：每个事务独立写日志和同步，磁盘同步延迟直接进入提交路径。

11.2 重构总原则

后续优化围绕五句话展开：

少做无效工作；冲突尽量提前；物理修改尽量批量；共享协调尽量局部；持久化同步尽量共享。

对应系统结构变化为：

路径	早期方式	当前方式	主要收益
请求入口	重复解析 SQL、较多独立往返	预处理计划复用 + 批执行	降低解析、规划和网络固定成本
事务写入	DML 较早修改 Heap/Index	事务私有暂存写	失败事务直接丢弃未应用写，减少重型回滚
并发控制	较粗粒度共享状态	256 分片多版本状态 + 64 分片缓冲页表	无关事务更容易并行
物理存储	逐记录页面访问、索引逐键下降	堆页批写 + B+ 树叶批写	摊薄页面、门锁和树遍历固定成本
日志提交	每事务独立同步	日志批量写 + 组提交	多事务共享稳定化成本
崩溃恢复	大量逐条小读取	固定缓冲顺序扫描	显著减少启动恢复系统调用

11.3 事务级合并优于单次操作微优化

写批不仅推迟物理修改，还会合并同一事务内连续变化。这意味着我们提交的是“事务最终需要产生的净变化”，而不是机械重放事务内部每一步临时状态。

这种设计同时影响 CPU、缓冲池、索引和日志：被抵消的逻辑写不再进入物理层，失败事务也不会先污染共享页面再执行昂贵撤销。

11.4 局部快速路径必须带安全回退

同页批写、同叶批写、索引叶页复用都属于性能快速路径，但它们必须建立在明确前置条件上。

例如索引叶页复用必须确认：

- 当前键仍属于该叶页；
- 叶页结构代际未发生变化；
- 页面仍有足够容量；
- 唯一性条件仍成立。

条件不满足时立即回退到完整 B+ 树流程，而不是假设并发结构没有变化。我们宁愿失去一次局部优化，也不让快速路径成为新的正确性分支。

11.5 性能和恢复共同演化

性能优化会改变系统其他部分的压力。前台吞吐提高后，WAL 量增大，原本可以接受的逐条恢复读取马上成为新的瓶颈。因此我们没有把恢复视为独立模块，而是把它看成高吞吐系统的组成部分。

同理，更大的缓冲池、更长的版本历史和事务写批会增加内存占用。性能提升不能只观察吞吐，还必须同时约束峰值内存、尾延迟、失败率和崩溃后就绪时间。

12 正确性设计

我们将系统正确性落实为一组在模块之间必须共同维持的不变量。

12.1 事务原子性

一个事务的堆表记录、所有相关索引、版本状态和事务元数据必须整体成功或整体失败。自动中止发生时，在向客户端返回失败状态之前必须完成相应回滚。

暂存写降低的是“尚未产生物理副作用”的回滚成本，并不改变事务整体回滚要求。

12.2 表与索引一致性

成功插入后，表记录和每个相关索引项必须同时存在；成功删除后，两者必须同时消失；更新索引列时，旧键删除与新键插入属于同一事务提交的一部分。

无论通过顺序扫描还是任何合法索引访问路径，在完成事务可见性和完整谓词判断后，都应得到同一个逻辑结果集合。

12.3 未提交数据不可见

事务自己的待提交版本通过私有覆盖实现读己之写，但不会直接成为其他事务可见的公共状态。其他事务只能读取其隔离级别允许的合法已提交版本。

12.4 无缺口快照

新事务只能捕获连续完整发布的提交前缀，不能跨过仍处于物理应用或持久化过程中的事务。这样可以避免快照同时看到部分新状态和部分旧状态。

12.5 预写日志先行

数据页被写回稳定存储之前，覆盖该页修改的日志必须已经持久化到对应位置。缓冲池在写出带有页面日志序列号的脏页前会先确保日志持久化水位满足要求。

12.6 提交成功之前必须持久化

向客户端返回提交成功之前，对应提交日志必须已经被实际稳定化操作覆盖。组提交可以共享一次同步，但不能提前确认某个尚未被持久化水位覆盖的事务。

12.7 恢复必须可重复

恢复过程中也可能再次发生崩溃。因此恢复不能依赖“这条日志以前一定没有执行过”的假设。记录安装、删除、索引维护和未提交事务撤销都按照可重复原则设计，使多次恢复最终收敛到相同合法状态。

12.8 浮点语义

TPC-C 金额路径对单精度浮点的存储、参数绑定、运算和聚合有严格要求。我们在执行更新和聚合时保持规定的 `binary32` / `binary64` 舍入边界，避免为了性能使用近似缓存或改变运算顺序所对应的外部语义。

12.9 装载与批处理仍属于事务语义

批量装载和网络批执行只是把多项工作集中处理，不代表可以跳过索引维护、日志、回滚或最终一致性。任何批量快速路径都必须在失败时恢复到完整旧状态。

13 性能结果与资源权衡

13.1 吞吐演进

我们的早期 32 客户端基线约为 **418 NewOrder/min**。随着事务私有写批、分片并发、批量物理提交、计划复用、组提交以及恢复路径重构逐步落地，最终正式三轮 NewOrder 吞吐为：

23,943.2 / 42,520.4 / 30,741.2 NewOrder/min

正式排名采用三轮中位数，因此最终结果为：

30,741.2 NewOrder/min

相较 418 的早期 32 客户端基线约提升 **73.5 倍**。

最终测量中 P50 延迟约 **13.218 ms**，P99 延迟约 **69.372 ms**。这说明系统从早期的并发退化状态转变为能够在多客户端下保持较高成功提交速率的架构。

13.2 性能提升的来源

我们不把最终吞吐归因于某一个函数或某一个参数。性能提升来自整个请求链中固定成本和共享等待的同时下降：

```
SQL 固定成本下降
+
无效事务工作量下降
+
Heap 页面访问次数下降
+
B+ 树重复下降次数下降
+
全局共享状态竞争下降
+
日志同步次数下降
=
端到端吞吐提升
```

其中最关键的架构变化是把旧系统的“全局等待、晚冲突、逐记录写入”改造成“分片并发、早冲突、事务级批提交”。

13.3 内存代价

更高并发度需要更大的缓冲池、事务私有写批、版本元数据和缓存结构，因此最终版本的峰值内存明显高于早期版本。

我们对缓冲池容量设置与物理内存和评测上限相关的约束，并控制多版本回收和提交并发度。在正式结果中峰值学生进程树 RSS 约为 **5.32 GiB**，仍处于允许范围。

因此当前架构是用一部分可控内存换取更高的页面命中率、事务并发度和批处理能力。

13.4 剩余性能瓶颈

从最终运行结果看，系统仍存在热点写冲突、日志同步和尾延迟问题。NewOrder、Delivery 等事务会集中访问部分 district、stock 和订单队列，高并发下真正的数据冲突无法通过分片消除。

日志持久化已经通过组提交摊销，但稳定存储同步仍是提交路径中不可消除的硬成本。未来需要在不提前确认提交的前提下继续优化日志批量和后台写回策略。

14 当前局限与后续方向

14.1 基于统计信息的成本优化器

当前查询优化主要依靠谓词下推、等值传播、连续复合索引前缀和启发式连接排序，尚未建立完整的表基数、列分布和选择率统计模型。

对于数据高度偏斜或多个索引选择率差异很大的场景，启发式计划不一定是最优的。后续可以引入统计信息、基数估计和成本模型，在多个合法计划之间进行更精确选择。

14.2 增量或模糊检查点

当前检查点仍需要较明显地协调事务和持久化状态。未来可以考虑模糊检查点或增量检查点，让正常事务与检查点过程更充分地并行，并进一步缩短日志重放范围。

14.3 多版本垃圾回收

旧版本什么时候可以回收取决于当前仍存在的最老快照。如果长事务长期持有旧快照，版本链就不能立即清理。

未来可以进一步优化版本垃圾回收索引、分批清理和长事务治理，减少历史版本占用的内存。

14.4 更完善的缓冲替换策略

当前替换器采用 LRU 类策略，结构简单且稳定，但大型顺序扫描仍可能对热点页面造成一定缓存污染。未来可以引入更适合数据库场景的访问频率/最近性混合策略，并结合扫描提示区分一次性页面与长期热点页面。

14.5 更细粒度的后台持久化

当前提交必须严格遵守预写日志和提交持久化边界，但脏页何时后台写回、多个日志段如何组织、同步请求如何与设备能力匹配仍有进一步优化空间。

14.6 索引结构维护

B+ 树的并发分裂、合并和惰性删除已经采用局部保护和安全回退，但更激进的后台合并、空间回收和并行结构维护仍需要在可验证的锁顺序和恢复语义基础上继续完善。

15 总结

我们最终形成的 RMDB 不再只是若干独立数据库模块的组合，而是一条贯通查询、事务、存储和恢复的完整执行链。

在读取路径上，我们通过语法与语义分析得到规范查询，利用谓词下推、等值传播、复合索引前缀和连接排序尽量减少候选记录，再通过顺序扫描或 B+ 树索引定位 Rid，最终结合堆表记录和事务快照得到真实可见结果。

在写入路径上，我们将 INSERT、UPDATE、DELETE 首先记录为事务私有暂存写，在冲突校验通过后才进入物理提交。提交时按堆表数据页和 B+ 树叶页聚合修改，减少重复页面访问和闕锁切换；多版本状态和缓冲页表分别采用分片结构，使不相关事务尽量在局部共享状态上并行。

在持久化路径上，我们使用预写日志保证数据页写盘之前日志已经持久化，通过组提交共享磁盘同步成本，并坚持在物理应用和提交日志稳定化完成后才发布事务为可见状态。

在崩溃恢复路径上，我们在数据库对外监听前完成日志分析、已提交事务的幂等重做和未提交事务的逆序撤销；通过固定缓冲区顺序扫描日志、复用分析结果、索引快照和定

向刷盘，将高吞吐带来的大日志量控制在可接受的启动恢复成本内。

整个优化过程的核心并不是让某一个算子“跑得更快”，而是重新组织事务工作发生的时间和共享范围：**能在事务私有状态完成的工作不提前进入共享存储；可以批量完成的物理操作不逐条执行；可以局部保护的数据结构不使用全局等待；已经确定失败的事务不继续消耗资源；可以共享的日志同步不让每个事务重复承担。**

这一设计使系统在保持事务、索引、日志和崩溃恢复语义完整的前提下，将 32 客户端 NewOrder 吞吐从早期的 418 NewOrder/min 提升到最终中位数 30,741.2 NewOrder/min，并为后续成本优化器、增量检查点、多版本垃圾回收和更精细的存储调度保留了清晰的演进空间。